

Каталог товаров с поиском

State • useState • Mock Data • Fetch API • Интерактивность

Дисциплина:	Frontend Development с React
Университет:	Технический Университет Молдовы
Лектор:	Александру Ярославски
Технологии:	Vite + React + TypeScript

Цель работы

Создать интерактивный каталог товаров с поиском, фильтрацией, кнопками действий и загрузкой данных из API. Сайт должен состоять из нескольких секций и реагировать на действия пользователя.

Обязательные разделы и функциональность

1. Header — Название магазина + навигация (Каталог, Избранное, О нас)
2. Hero / Баннер — Заголовок + описание магазина + кнопка «Перейти к каталогу»
3. Поиск — Поле ввода для поиска товаров в реальном времени (onChange + useState)
4. Каталог товаров — Сетка карточек товаров из Mock Data или API. Каждая карточка: изображение, название, цена, кнопка «В корзину» или «Like»
5. Фильтрация — Минимум 2 кнопки-фильтра по категориям (Все, Электроника, Одежда и т.д.)
6. Счётчик — Отображение количества найденных товаров и/или товаров в корзине
7. Footer — Информация о магазине, копирайт

Структура проекта

// Минимум 8 компонентов + 1 файл данных

```
src/  
App.tsx  
App.css  
data/  
products.ts // Mock Data — массив товаров  
components/  
Header.tsx  
Hero.tsx  
SearchBar.tsx // Поле поиска (props: value, onChange)  
FilterButtons.tsx // Кнопки фильтрации  
ProductCard.tsx // Карточка товара (props)  
ProductList.tsx // Сетка товаров (.map)  
Counter.tsx // Счётчик товаров/корзины  
Footer.tsx
```

УРОВЕНЬ 1: Базовый (оценка 5-7)

Минимальные требования:

- Массив Mock Data в файле products.ts (минимум 6 товаров)
- Поиск: input + useState + filter по названию
- Список товаров через .map() с карточками
- Каждая карточка содержит: название, цена, кнопку
- Минимум 5 секций на странице
- Базовая стилизация CSS
- Нет ошибок в консоли

Пример: products.ts (Mock Data)

// src/data/products.ts

```
export type Product = {
  id: number;
  name: string;
  price: number;
  category: string;
  image: string;
};

export const products: Product[] = [
  { id: 1, name: "Ноутбук", price: 15000, category: "electronics",
    image: "https://picsum.photos/seed/laptop/300/200" },
  { id: 2, name: "Футболка", price: 500, category: "clothes",
    image: "https://picsum.photos/seed/tshirt/300/200" },
  // ... минимум 6 товаров
];
```

Пример: Поиск в App.tsx (уровень 1)

// App.tsx – поиск + счётчик

```
import { useState } from "react";
import { products } from "../data/products";

function App() {
  const [search, setSearch] = useState<string>("");

  const filtered = products.filter(p =>
    p.name.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div>
      <input value={search} onChange={e => setSearch(e.target.value)}
        placeholder="Поиск товаров..." />
      <p>Найдено: {filtered.length} товаров</p>
      {filtered.map(p => <ProductCard key={p.id} {...p} />)}
    </div>
  );
}
```

УРОВЕНЬ 2: Продвинутый (оценка 8-10)

Всё из уровня 1 ПЛЮС:

- Кнопки фильтрации по категориям (Все, Электроника, Одежда...) с useState
- Кнопка Like / В корзину на каждой карточке с useState (счётчик лайков)
- Fetch данных из API вместо или в дополнение к Mock Data: fakestoreapi.com/products
- Loading state: показываем «Загрузка...» пока данные грузятся
- Error state: показываем сообщение при ошибке сети
- Empty state: «Ничего не найдено» когда фильтр пуст
- Адаптивная сетка CSS Grid / Flexbox для карточек
- Hover-эффекты на карточках

Пример: Fetch + Loading (уровень 2)

// Fetch + Loading + Error

```
const [products, setProducts] = useState<Product[]>([]);
const [loading, setLoading] = useState<boolean>(true);
const [error, setError] = useState<string | null>(null);

useEffect(() => {
  const load = async () => {
    try {
      const res = await fetch("https://fakestoreapi.com/products");
      if (!res.ok) throw new Error("Ошибка сервера");
      const data = await res.json();
      setProducts(data);
    } catch (err: any) { setError(err.message); }
    finally { setLoading(false); }
  };
  load();
}, []);

if (loading) return <p>Загрузка...</p>;
if (error) return <p>Ошибка: {error}</p>;
```

Пример: ProductCard с Like (уровень 2)

// ProductCard.tsx — с кнопкой Like

```
import { useState } from "react";

type Props = { name: string; price: number; image: string; };

function ProductCard({ name, price, image }: Props) {
  const [liked, setLiked] = useState<boolean>(false);

  return (
    <div className="product-card">
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p>{price} MDL</p>
      <button onClick={() => setLiked(!liked)}>
        {liked ? "♥ Liked" : "♥ Like"}
      </button>
    </div>
  );
}
```

ЛАБОРАТОРНАЯ 2

#	Вопрос	ответ
1	Что такое компонент в React?	
2	Чем .tsx отличается от .ts?	
3	Что такое Props?	
4	Зачем нужен key в .map()?	
5	Почему компонент пишется с большой буквы?	
6	Что делает export default?	
7	Чем className отличается от class?	

#	Вопрос	ответ
1	Что такое useState?	
2	Чем Props отличается от State?	
3	Что делает setSearch()?	
4	Зачем useEffect для fetch?	
5	Что означает [] в useEffect?	
6	Что такое Mock Data?	
7	Зачем проверять response.ok?	
8	Что такое async/await?	
9	Зачем loading state?	
10	Что такое условный рендеринг?	